

Kali Linux como plataforma práctica para auditorías de seguridad y aprendizaje automatizable

Resumen

Este informe analiza Kali Linux como distribución orientada a seguridad informática y como entorno práctico para organizar pruebas controladas. Se presenta primero una visión general de familias de herramientas disponibles en Kali, cubriendo reconocimiento, análisis web, enumeración, explotación controlada, tráfico de red, contraseñas, análisis forense básico y OSINT. A continuación, se desarrolla una parte práctica estructurada como una auditoría red team básica sobre un entorno propio local, con evidencias generadas mediante Nmap, WhatWeb, ffuf, sqlmap, tcpdump, John the Ripper y exiftool.

Se han incorporado scripts como apoyo complementario para repetir comandos y conservar evidencias. De forma separada, la idea de crear un prototipo alpha de agente asistido en terminal actuó como incentivo personal para mantener el interés en la parte práctica. Ese prototipo se llegó a probar sobre un flujo web vulnerable local con Nmap, WhatWeb, ffuf y sqlmap, y se presenta como una exploración motivadora del aprendizaje, no como herramienta final. Durante la investigación asociada a esta idea se identificó HexStrike AI, herramienta reciente del ecosistema Kali, que se menciona como ejemplo actual relacionado con agentes y automatización en seguridad.

1. Introducción

Kali Linux es una distribución orientada al trabajo técnico en seguridad informática. Su valor principal está en ofrecer un entorno preparado para tareas habituales de auditoría, análisis, documentación y aprendizaje práctico (1). En el contexto de este trabajo, se estudia Kali como una distribución basada en seguridad y como plataforma para organizar pequeñas pruebas controladas.

El objetivo del informe es analizar herramientas representativas de distintos ámbitos: reconocimiento de red, análisis web, enumeración, explotación controlada, captura de tráfico, auditoría de contraseñas y análisis forense básico. El trabajo se centra en un conjunto de herramientas que permite explicar finalidad, configuración, ejecución, salida obtenida e interpretación.

Por este motivo, se ha optado por incluir una parte práctica en la que las pruebas se articulan como una auditoría red team básica sobre un entorno propio local. Esta elección mantiene el objetivo académico del trabajo, centrado en conocer y comparar herramientas de Kali, y al mismo tiempo ofrece un hilo conductor más motivador: cada herramienta se aplica en una fase

concreta y genera una evidencia verificable. Los scripts ayudan a repetir esas ejecuciones y conservar resultados; la propuesta de un prototipo de agente asistido queda como un elemento adicional que sirvió de incentivo personal para sostener el avance del proyecto.

2. Contexto de Kali Linux

Kali Linux es una distribución GNU/Linux basada en Debian y enfocada en seguridad ofensiva, análisis, auditoría y formación. Su uso habitual se relaciona con laboratorios de ciberseguridad, pruebas de penetración autorizadas, análisis forense, ingeniería inversa, auditoría inalámbrica y evaluación de configuraciones (1, 2).

Una característica relevante de Kali es que integra o facilita el acceso a herramientas especializadas que, en una distribución generalista, tendrían que instalarse y configurarse por separado (2). Esto permite que el entorno de trabajo esté orientado desde el inicio a tareas de seguridad. La práctica responsable sitúa esas herramientas en sistemas propios o en entornos donde existe autorización explícita.

En este trabajo, Kali se emplea como sistema desde el que se ejecutan prácticas controladas y se recogen evidencias. El foco está en mostrar cómo diferentes familias de herramientas aportan información técnica en fases concretas de una auditoría básica.

3. Distribuciones de seguridad y Kali Linux

Las distribuciones de seguridad son sistemas operativos preparados para tareas específicas de análisis, auditoría o respuesta técnica. A diferencia de una distribución de propósito general, suelen incluir herramientas preinstaladas, configuraciones orientadas al trabajo técnico y documentación relacionada con escenarios de seguridad.

Dentro de este grupo, Kali Linux ocupa un lugar destacado por su orientación práctica y por la amplitud de herramientas disponibles (1, 2). Su enfoque permite cubrir desde tareas iniciales, como descubrir servicios activos, hasta fases más avanzadas, como analizar tráfico, probar credenciales en laboratorio o estudiar archivos desde una perspectiva forense.

Kali proporciona el entorno y las herramientas; la interpretación de resultados se apoya en el conocimiento técnico, el contexto de la prueba y la autorización del entorno.

4. Familias de herramientas disponibles en Kali

Kali reúne herramientas orientadas a distintas fases de una auditoría. Algunas se usan para descubrir sistemas y servicios, otras para analizar aplicaciones web, capturar tráfico, evaluar contraseñas, estudiar archivos o recopilar información pública. La tabla resume herramientas representativas y permite comparar su función dentro del flujo de trabajo.

Herramienta	Finalidad	Ámbito de seguridad	Tipo de salida	Ventajas	Limitaciones prácticas
Nmap	Descubrir hosts, puertos, servicios y versiones	Reconocimiento de red	Texto, XML, grepable	Muy completo, documentable y flexible	Requiere interpretación de puertos, servicios y contexto
Netdiscover	Detectar equipos en una red local mediante ARP	Reconocimiento de red	Tabla de IP, MAC y fabricante	Rápido para redes locales	Centrado LAN y tráfico ARP
Masscan	Escanear puertos a gran velocidad	Reconocimiento de red	Texto, JSON, formatos compatibles	Muy rápido en rangos amplios	Exige ajustes de ritmo y alcance con cuidado
WhatWeb	Identificar tecnologías visibles en una web	Análisis web	Texto con plugins detectados	Rápido y útil para huella tecnológica	Depende de lo que exponga la respuesta HTTP
ffuf	Enumerar rutas, parámetros o valores con wordlists	Enumeración web	Texto, JSON, HTML	Rápido, flexible y automatizable	La calidad depende de la wordlist y filtros
Gobuster	Enumerar directorios, DNS o vhosts	Enumeración web	Texto estructurado	Sencillo y muy conocido	Más flexible que ffuf en algunos escenarios
Burp Suite	Interceptar, modificar y analizar tráfico web	Auditoría web	Peticiones, respuestas, hallazgos, historial	Muy útil para análisis manual	Requiere navegación configurada para práctica
OWASP ZAP	Proxy y escáner web orientado a seguridad	Auditoría web y blue team	Alertas, reportes, historial HTTP	Abierto, visual y adecuado para aprendizaje	Puede generar resultados que requieren verificación

Herramienta	Finalidad	Ámbito de seguridad	Tipo de salida	Ventajas	Limitaciones prácticas
sqlmap	Detectar y explotar inyecciones SQL autorizadas	Explotación controlada	Texto, sesiones y resultados por objetivo	Automatiza pruebas complejas de SQLi	Debe acotarse a laboratorio autorizado expresa
Metasploit	Usar módulos de explotación, prueba y post-explotación	Explotación controlada	Consola, sesiones, reportes auxiliares	Marco amplio y modular	Puede alargar mucho una práctica
tcpdump	Capturar paquetes desde terminal	Análisis de tráfico	.pcap y texto	Ligero, scriptable y preciso	Suele requerir permisos de captura
Wireshark	Inspeccionar tráfico de red visualmente	Análisis de tráfico y blue team	.pcap, filtros, vistas de protocolo	Muy claro para análisis visual	Interfaz amplia y curva de filtros
John the Ripper	Auditar hashes con diccionarios y reglas	Contraseñas	Texto, potfile, resultados --show	Práctico para explicar ataques de diccionario	Depende de formato y de los candidatos
Hashcat	Auditar hashes con aceleración por GPU	Contraseñas	Texto, sesiones, resultados recuperados	Muy potente y rápido con hardware adecuado	Configura y rendimiento dependen del equipo
Hydra	Probar credenciales contra servicios autorizados	Contraseñas y autenticación	Texto con intentos y credenciales válidas	Soporta muchos protocolos	Requiere control estricto de alcance y ritmo
exiftool	Extraer metadatos de archivos	Forense básico	Texto con propiedades y metadatos	Claro, rápido y seguro para evidencias simples	La utilidad depende del tipo de archivo

Herramienta	Finalidad	Ámbito de seguridad	Tipo de salida	Ventajas	Limitaciones prácticas
binwalk	Analizar firmware y binarios buscando estructuras	Forense y análisis de archivos	Texto, extracción de bloques	Útil para firmware y archivos embebidos	Más técnicas especializadas
Autopsy	Analizar imágenes forenses y sistemas de archivos	Forense digital	Interfaz, casos, reportes	Completo para investigación forense	Requiere preparar casos y evidencias
theHarvester	Recopilar información pública de dominios	OSINT	Dominios, correos, hosts, fuentes	Sencillo para primeras búsquedas OSINT	Depende de fuentes externas y cuotas
Maltego	Representar relaciones entre entidades OSINT	OSINT y análisis visual	Grafos, entidades, transformadas	Muy visual para relaciones complejas	Requiere configuración de transformaciones

4.1 Reconocimiento y escaneo de red

El reconocimiento de red busca identificar sistemas, puertos abiertos, servicios y versiones. Nmap destaca por su equilibrio entre profundidad y documentación de resultados (3, 4). Netdiscover aporta descubrimiento rápido en redes locales mediante ARP, útil para ubicar equipos activos. Masscan se orienta a escaneos de alta velocidad y resulta interesante cuando el alcance es amplio y está correctamente delimitado.

4.2 Análisis y enumeración web

El análisis web permite conocer tecnologías, cabeceras, rutas y posibles puntos de entrada de una aplicación. WhatWeb ofrece una identificación rápida de tecnologías visibles (5). ffuf y Gobuster cubren enumeración mediante wordlists, con ffuf como opción flexible para fuzzing y Gobuster como alternativa sencilla. Burp Suite y OWASP ZAP añaden una capa interactiva mediante proxy, historial de peticiones y análisis de respuestas, con especial utilidad para revisión manual y aprendizaje.

4.3 Explotación controlada

La explotación controlada comprueba vulnerabilidades dentro de un entorno autorizado. sqlmap se centra en inyección SQL y permite una práctica acotada contra una aplicación

vulnerable local (8). Metasploit Framework ofrece un marco más amplio de módulos para pruebas de seguridad, explotación y validación de configuraciones, adecuado para laboratorios con objetivos preparados.

4.4 Análisis de tráfico

El análisis de tráfico permite observar comunicaciones de red, protocolos, direcciones, puertos y paquetes intercambiados. tcpdump representa el enfoque de línea de comandos: ligero, automatizable y adecuado para guardar capturas en formato .pcap (12). Wireshark complementa esa captura con una interfaz visual, filtros de protocolo y vistas detalladas que facilitan el análisis defensivo y la explicación de paquetes.

4.5 Auditoría de contraseñas

Las herramientas de contraseñas permiten evaluar hashes o mecanismos de autenticación en entornos autorizados. John the Ripper trabaja muy bien con hashes y wordlists en prácticas pequeñas (9). Hashcat amplía ese enfoque con aceleración por GPU y un rendimiento elevado. Hydra se orienta a pruebas de autenticación contra servicios concretos, por lo que su uso se vincula a objetivos propios, límites claros y ritmos controlados.

4.6 Análisis forense básico

El análisis forense básico extrae información de archivos, soportes o evidencias digitales. exiftool resulta adecuado para una práctica breve porque muestra metadatos de forma clara: tipo, tamaño, fechas, codificación, permisos y otros atributos (10). binwalk se orienta a firmware, binarios y estructuras embebidas. Autopsy proporciona un entorno completo para casos forenses con imágenes, sistemas de archivos y reportes.

4.7 OSINT y recopilación de información

OSINT se refiere a la recopilación de información desde fuentes abiertas. theHarvester permite obtener dominios, correos o hosts asociados a un objetivo público. Maltego representa relaciones entre entidades mediante grafos y transformadas. En el panorama de Kali, estas herramientas completan la visión de fases previas de información y análisis visual.

4.8 Automatización y scripting

La automatización aparece en este trabajo como apoyo práctico complementario. Se usan scripts Bash sencillos en `lab/scripts/` para fijar los comandos ejecutados, pedir los argumentos necesarios y guardar las salidas en `lab/resultados/`. Su función es reproducir las pruebas en condiciones similares y conservar evidencias de forma ordenada.

5. Criterios de selección de herramientas prácticas

Tras presentar el panorama general de herramientas, la parte práctica se concentra en una selección manejable y representativa. El criterio principal ha sido combinar cobertura y viabilidad: cada herramienta debía pertenecer a un ámbito claro de la seguridad, poder ejecutarse en un entorno local controlado y generar una salida técnica que pudiera guardarse como evidencia.

Con estos criterios, se han seleccionado Nmap para reconocimiento de red, WhatWeb para identificación web, ffuf para enumeración, sqlmap para explotación controlada, tcpdump para análisis de tráfico, John the Ripper para auditoría de contraseñas y exiftool para análisis forense básico. En conjunto, estas herramientas permiten recorrer fases diferentes de una auditoría red team básica y documentar en cada caso la configuración, la ejecución, la salida obtenida y su interpretación.

La seguridad del laboratorio también forma parte de la selección. Las pruebas se han realizado sobre servicios propios, archivos de prueba, hashes creados para el trabajo y aplicaciones vulnerables preparadas localmente. Cuando una herramienta requiere permisos especiales, ese requisito se documenta junto al comando utilizado para que la evidencia quede vinculada al contexto real de ejecución.

6. Entorno de laboratorio

El laboratorio se organiza como un conjunto de escenarios propios y controlados, cada uno asociado a una herramienta concreta. Esta estructura permite mantener la práctica dentro de un alcance seguro y, al mismo tiempo, obtener evidencias técnicas suficientes para explicar el funcionamiento de cada herramienta. Para ello se prepararon una web estática, una aplicación vulnerable mínima para sqlmap, hashes de prueba, una wordlist reducida y un archivo propio de evidencia.

Antes de ejecutar las pruebas se documentó el entorno base. La información del sistema se guardó en `lab/resultados/entorno_uname.txt`, `lab/resultados/entorno_os_release.txt` y `lab/resultados/entorno_ip_addr.txt`. El sistema usado se identifica como Kali GNU/Linux Rolling 2026.1, y los escenarios se apoyan principalmente en la interfaz loopback `127.0.0.1`.

La web local de prueba se sirvió en `127.0.0.1:8080` mediante el servidor HTTP de Python. La práctica de sqlmap utilizó una aplicación vulnerable mínima en `127.0.0.1:8090`, implementada en Python y SQLite. La captura con tcpdump se realizó sobre la interfaz loopback, generando tráfico local para obtener un archivo `.pcap` revisable posteriormente.

La siguiente tabla resume las evidencias técnicas generadas:

Ámbito	Herramienta	Evidencia principal	Estado
Entorno	Comandos del sistema	lab/resultados/entorno_uname.txt , lab/resultados/entorno_os_release.txt , lab/resultados/entorno_ip_addr.txt	Generado
Reconocimiento	Nmap	lab/resultados/nmap_recon_127.0.0.1.txt	Generado
Identificación web	WhatWeb	lab/resultados/whatweb_127.0.0.1_8080.txt	Generado
Enumeración web	ffuf	lab/resultados/ffuf_127.0.0.1_8080_fuzz.json	Generado
Inyección SQL controlada	sqlmap	lab/resultados/sqlmap/sqlmap_resumen.txt	Generado
Captura de tráfico	tcpdump	lab/resultados/tcpdump_lo_20s.pcap	Generado
Auditoría de hashes	John the Ripper	lab/resultados/john_show_hashes_prueba.txt	Generado
Metadatos	exiftool	lab/resultados/exiftool_archivo_prueba.txt	Generado

[INSERTAR CAPTURA: lab/capturas/entorno_kali.png , entorno de Kali Linux utilizado para las pruebas]

7. Casos prácticos con herramientas seleccionadas

Los resultados de esta sección proceden de evidencias guardadas en lab/resultados/ y de archivos de apoyo situados en lab/evidencias/. Las marcas de captura indican el punto exacto donde insertar las imágenes tomadas durante la ejecución.

7.1 Reconocimiento de red con Nmap

Finalidad: Nmap se utiliza para descubrir puertos abiertos y obtener información sobre los servicios expuestos. En una auditoría real ayuda a conocer la superficie visible de un sistema antes de realizar pruebas más concretas.

Configuración usada: se levantó una web local de prueba en 127.0.0.1:8080 mediante el servidor HTTP de Python y se escaneó la dirección loopback 127.0.0.1. La opción principal fue -sV , que intenta identificar versiones de servicios.

Comando base:

```
nmap -sV -oN lab/resultados/nmap_recon_127.0.0.1.txt 127.0.0.1
```

Lectura del comando: `-sV` activa la detección de versiones de servicios, `-oN` guarda la salida en formato normal y `127.0.0.1` dirige la prueba a la propia máquina. Esta configuración demuestra un reconocimiento básico sobre un objetivo local.

Script de apoyo:

```
./lab/scripts/nmap_recon.sh 127.0.0.1
```

El script recibe el objetivo como argumento, crea `lab/resultados/` si no existe y ejecuta el mismo comando con salida normal. Su aportación está en fijar los parámetros usados en el laboratorio y hacer repetible el guardado de la evidencia.

Resultado e interpretación: el resultado quedó guardado en `lab/resultados/nmap_recon_127.0.0.1.txt`. La salida detectó el puerto `8080/tcp` abierto con un servicio HTTP asociado a `SimpleHTTPServer 0.6 (Python 3.13.12)`. En la ejecución final, al estar levantada también la aplicación vulnerable de `sqlmap`, apareció además el puerto `8090/tcp` asociado a `BaseHTTPServer 0.6 (Python 3.13.12)`. Esto confirma que los servicios locales preparados estaban activos y visibles desde la propia máquina.

[INSERTAR CAPTURA: `lab/capturas/nmap_servicio_local.png`, ejecución de Nmap mostrando puertos y servicios detectados]

7.2 Análisis web con WhatWeb

Finalidad: WhatWeb identifica tecnologías visibles en una aplicación web, como servidor, cabeceras, título HTML o características detectables desde la respuesta HTTP. Sirve como primera huella tecnológica del objetivo.

Configuración usada: se analizó la URL `http://127.0.0.1:8080`, correspondiente a la web local del laboratorio. Se usó `--color=never` para que la salida guardada no incluya códigos de color de terminal.

Comando base:

```
whatweb --color=never http://127.0.0.1:8080
```

Lectura del comando: `--color=never` guarda el resultado como texto limpio y la URL indica el servicio web local analizado. La herramienta realiza una petición HTTP y extrae tecnologías reconocibles de la respuesta.

Script de apoyo:

```
./lab/scripts/whatweb_scan.sh http://127.0.0.1:8080
```

El script toma la URL como argumento, genera un nombre de archivo a partir del objetivo y guarda la salida con `tee`, de forma que se ve en pantalla y queda registrada en `lab/resultados/`.

Resultado e interpretación: el resultado quedó guardado en `lab/resultados/whatweb_127.0.0.1_8080.txt`. La salida identificó HTML5, servidor SimpleHTTP/0.6 Python/3.13.12, dirección 127.0.0.1 y el título Laboratorio local Kali. Esto demuestra que la herramienta puede extraer información tecnológica básica sin necesidad de interactuar manualmente con el navegador.

[INSERTAR CAPTURA: `lab/capturas/whatweb_web_local.png`, ejecución de WhatWeb sobre el objetivo web de laboratorio]

7.3 Enumeración web con ffuf

Finalidad: ffuf se utiliza para enumerar rutas, directorios o parámetros mediante fuzzing con wordlists. En este laboratorio se usa para descubrir rutas web existentes en una aplicación controlada.

Configuración usada: se creó una estructura local bajo `lab/evidencias/web_local/` con rutas controladas y una wordlist reducida en `lab/evidencias/wordlists/web_local.txt`. La URL incluye la palabra `FUZZ`, que ffuf sustituye por cada entrada de la wordlist.

Comando base:

```
ffuf -u http://127.0.0.1:8080/FUZZ -w lab/evidencias/wordlists/web_local.txt -  
of json -o lab/resultados/ffuf_127.0.0.1_8080_fuzz.json
```

Lectura del comando: `-u` define la URL que contiene la marca `FUZZ`, `-w` indica la wordlist, `-of json` fija el formato de salida y `-o` guarda el resultado. La configuración permite repetir la enumeración con una lista pequeña y controlada.

Script de apoyo:

```
./lab/scripts/ffuf_enum.sh http://127.0.0.1:8080/FUZZ  
lab/evidencias/wordlists/web_local.txt
```

El script valida que la wordlist exista y que la URL contenga `FUZZ`. Esta validación prepara una ejecución coherente y deja claro qué parte de la URL será sustituida por los candidatos.

Resultado e interpretación: el resultado quedó guardado en

lab/resultados/ffuf_127.0.0.1_8080_fuzz.json . La ejecución encontró las rutas `admin` , `login` y `assets` , todas con respuesta HTTP 301 hacia su forma con barra final. Esto indica que las rutas existen en el servidor y que el servidor redirige a la versión terminada en `/` .

[INSERTAR CAPTURA: lab/capturas/ffuf_web_local.png , ejecución de ffuf con wordlist controlada]

7.4 Explotación controlada con sqlmap

Finalidad: sqlmap automatiza la detección y explotación controlada de inyecciones SQL. Su uso requiere un sistema propio o con autorización explícita porque puede generar consultas invasivas.

Configuración usada: se preparó una aplicación vulnerable local en

lab/evidencias/sqlmap_lab/ , accesible en 127.0.0.1:8090 . La aplicación expone el endpoint `http://127.0.0.1:8090/product?id=1` , usa SQLite y contiene una vulnerabilidad intencionada al concatenar directamente el parámetro `id` en una consulta SQL. La prueba se acotó con `--level=1` y `--risk=1` .

Comando base:

```
sqlmap -u 'http://127.0.0.1:8090/product?id=1' --batch --level=1 --risk=1 --  
output-dir=lab/resultados/sqlmap
```

Lectura del comando: `-u` fija la URL vulnerable, `--batch` automatiza respuestas por defecto, `--level=1` y `--risk=1` mantienen una prueba acotada, y `--output-dir` centraliza la salida. La prueba se dirige al parámetro `id` de una aplicación vulnerable local.

Script de apoyo:

```
./lab/scripts/sqlmap_lab.sh 'http://127.0.0.1:8090/product?id=1'
```

El script recibe la URL como argumento, fija el directorio de salida y ejecuta sqlmap en modo acotado con `--level=1` y `--risk=1` .

Resultado e interpretación: el resultado quedó guardado en

lab/resultados/sqlmap/sqlmap_resumen.txt . sqlmap identificó el parámetro GET `id` como vulnerable y detectó SQLite como sistema gestor de base de datos. La herramienta reportó técnicas de inyección boolean-based blind, time-based blind y UNION query. Esto confirma que la vulnerabilidad preparada en la aplicación local es detectable por una herramienta automatizada.

[INSERTAR CAPTURA: lab/capturas/sqlmap_laboratorio.png , ejecución de sqlmap acotada al entorno de laboratorio]

7.5 Análisis de tráfico con tcpdump

Finalidad: tcpdump permite capturar paquetes de red desde la línea de comandos y guardarlos en formato .pcap para análisis posterior. Es útil para comprobar tráfico real generado por una prueba.

Configuración usada: captura en la interfaz lo durante una ventana breve de 20 segundos, generando tráfico propio contra la web local.

Comando base:

```
sudo tcpdump -i lo -w lab/resultados/tcpdump_lo_20s.pcap
```

Lectura del comando: sudo es necesario cuando el usuario no tiene permisos de captura, -i lo selecciona la interfaz loopback y -w escribe los paquetes en un archivo .pcap . El archivo resultante puede abrirse después con herramientas como Wireshark o analizarse con tcpdump.

Generación de tráfico:

```
curl http://127.0.0.1:8080
```

Script de apoyo:

```
./lab/scripts/tcpdump_capture.sh lo 20
```

El script recibe la interfaz y la duración como argumentos. Su finalidad es acotar la captura a una ventana corta y cerrar el proceso al terminar el tiempo indicado.

Resultado e interpretación: se generó el archivo lab/resultados/tcpdump_lo_20s.pcap , con un tamaño de 27346 bytes. La lectura del archivo con tcpdump -r mostró 227 paquetes IP en loopback entre direcciones 127.0.0.1 , lo que confirma que la captura registró tráfico local real. El archivo está en formato .pcap , por lo que puede revisarse posteriormente con tcpdump o Wireshark.

[INSERTAR CAPTURA: lab/capturas/tcpdump_trafico_local.png , ejecución de tcpdump capturando tráfico del laboratorio]

7.6 Auditoría de contraseñas con John the Ripper

Finalidad: John the Ripper permite auditar hashes de contraseñas probando candidatos de una wordlist. En este trabajo se usa con un hash de prueba creado para la práctica.

Configuración usada: hash MD5 de prueba en `lab/evidencias/hashes_prueba.txt` , wordlist local en `lab/evidencias/wordlist_hashes_prueba.txt` y formato Raw-MD5 .

Funcionamiento: John trabaja por comparación. Calcula el hash de cada contraseña candidata de la wordlist y lo compara con el hash objetivo. Si ambos valores coinciden, puede mostrar la contraseña candidata que originó ese hash.

Comando base:

```
john --format=Raw-MD5 --wordlist=lab/evidencias/wordlist_hashes_prueba.txt  
lab/evidencias/hashes_prueba.txt
```

Lectura del comando: `--format=Raw-MD5` indica el tipo de hash, `--wordlist` proporciona los candidatos y el último argumento es el archivo de hashes. Esta configuración permite comprobar si la contraseña de prueba aparece en la lista.

Script de apoyo:

```
./lab/scripts/john_hash.sh lab/evidencias/hashes_prueba.txt  
lab/evidencias/wordlist_hashes_prueba.txt
```

El script configura un archivo `pot` propio en `lab/resultados/` , ejecuta John y después lanza `john --show` para mostrar el resultado recuperado. Así queda documentado tanto el intento de auditoría como la contraseña encontrada.

Resultado e interpretación: el resultado quedó guardado en `lab/resultados/john_show_hashes_prueba.txt` . John the Ripper recuperó la contraseña de prueba `password` para el hash preparado. Esto demuestra el funcionamiento básico de un ataque de diccionario sobre hashes, sin emplear credenciales reales.

[INSERTAR CAPTURA: `lab/capturas/john_hashes_prueba.png` , ejecución de John the Ripper mostrando el uso de una wordlist local]

7.7 Análisis forense básico con exiftool

Finalidad: exiftool extrae metadatos de archivos. En análisis forense básico permite revisar tipo de archivo, tamaño, fechas, codificación y otros atributos sin abrir el contenido de forma manual.

Configuración usada: archivo propio `lab/evidencias/archivo_prueba.txt` .

Comando base:

```
exiftool lab/evidencias/archivo_prueba.txt
```

Lectura del comando: exiftool recibe directamente la ruta del archivo y devuelve metadatos técnicos sin modificar la evidencia. En este caso se trabaja con un archivo de texto propio creado para el laboratorio.

Script de apoyo:

```
./lab/scripts/exiftool_meta.sh lab/evidencias/archivo_prueba.txt
```

El script valida que el archivo exista, calcula un nombre de salida basado en el nombre del archivo y guarda los metadatos en `lab/resultados/`.

Resultado e interpretación: el resultado quedó guardado en `lab/resultados/exiftool_archivo_prueba.txt`. La salida identificó el archivo como texto plano ASCII, con 128 bytes, 3 líneas y 19 palabras. Esto muestra cómo exiftool resume propiedades técnicas útiles para documentar una evidencia sencilla.

[INSERTAR CAPTURA: `lab/capturas/exiftool_archivo_prueba.png`, ejecución de exiftool mostrando metadatos extraídos]

8. Scripts de apoyo para reproducibilidad

Los scripts incluidos en `lab/scripts/` tienen una justificación práctica limitada y concreta: reproducir las ejecuciones del laboratorio y conservar evidencias con nombres y rutas estables. Su función es recoger los comandos usados durante las prácticas para poder repetirlos en las mismas condiciones, manteniendo el protagonismo técnico en las herramientas de Kali.

La estructura común es sencilla. Cada script recibe los argumentos necesarios, comprueba condiciones básicas cuando procede, crea `lab/resultados/` y ejecuta la herramienta correspondiente guardando la salida. Esto resulta útil en un trabajo con varias prácticas porque reduce errores de rutas, evita mezclar resultados y facilita relacionar cada explicación con un archivo concreto.

El script `lab/scripts/nmap_recon.sh` sirve como ejemplo. Recibe un objetivo, prepara el directorio de resultados y ejecuta Nmap con la misma opción usada en la práctica:

```
nmap -sV -oN "$SALIDA_DIR/nmap_recon_${OBJETIVO}.txt" "$OBJETIVO"
```

Los demás scripts aplican la misma idea a WhatWeb, ffuf, sqlmap, tcpdump, John the Ripper y exiftool. En conjunto, ofrecen una forma sencilla de repetir las prácticas y mantener las evidencias asociadas a cada herramienta.

9. Prototipo de agente asistido como incentivo de trabajo

La idea de desarrollar un prototipo de agente asistido forma una parte complementaria del trabajo. Su interés principal dentro del proyecto es personal y metodológico: sirvió como incentivo para mantener la motivación y sostener el objetivo principal, que es conocer Kali Linux y sus herramientas de seguridad.

El prototipo se orienta a actuar desde la terminal de Kali sobre acciones previamente autorizadas. Para ello se plantea una política local donde se declaran comandos permitidos, objetivos autorizados, rutas de escritura y límites técnicos. Sobre esa base, el agente organiza un proceso por fases: objetivo, validación, ejecución, evidencia y resumen. La idea se presenta como una prueba alpha nacida del interés personal por explorar una forma más atractiva de recorrer la práctica.

En la versión probada, el agente dispone de una interfaz en terminal. El usuario selecciona el perfil de análisis `Web vulnerable`, introduce la dirección objetivo, elige una wordlist para ffuf y decide si añade una comprobación con sqlmap sobre una URL vulnerable con parámetro. Tras la confirmación escrita de autorización, el agente genera un plan de sesión y ejecuta fases con herramientas ya explicadas en el informe: Nmap, WhatWeb, ffuf y, de forma opcional, sqlmap. Al terminar, muestra los resultados de cada herramienta y guarda el resumen de ejecución.

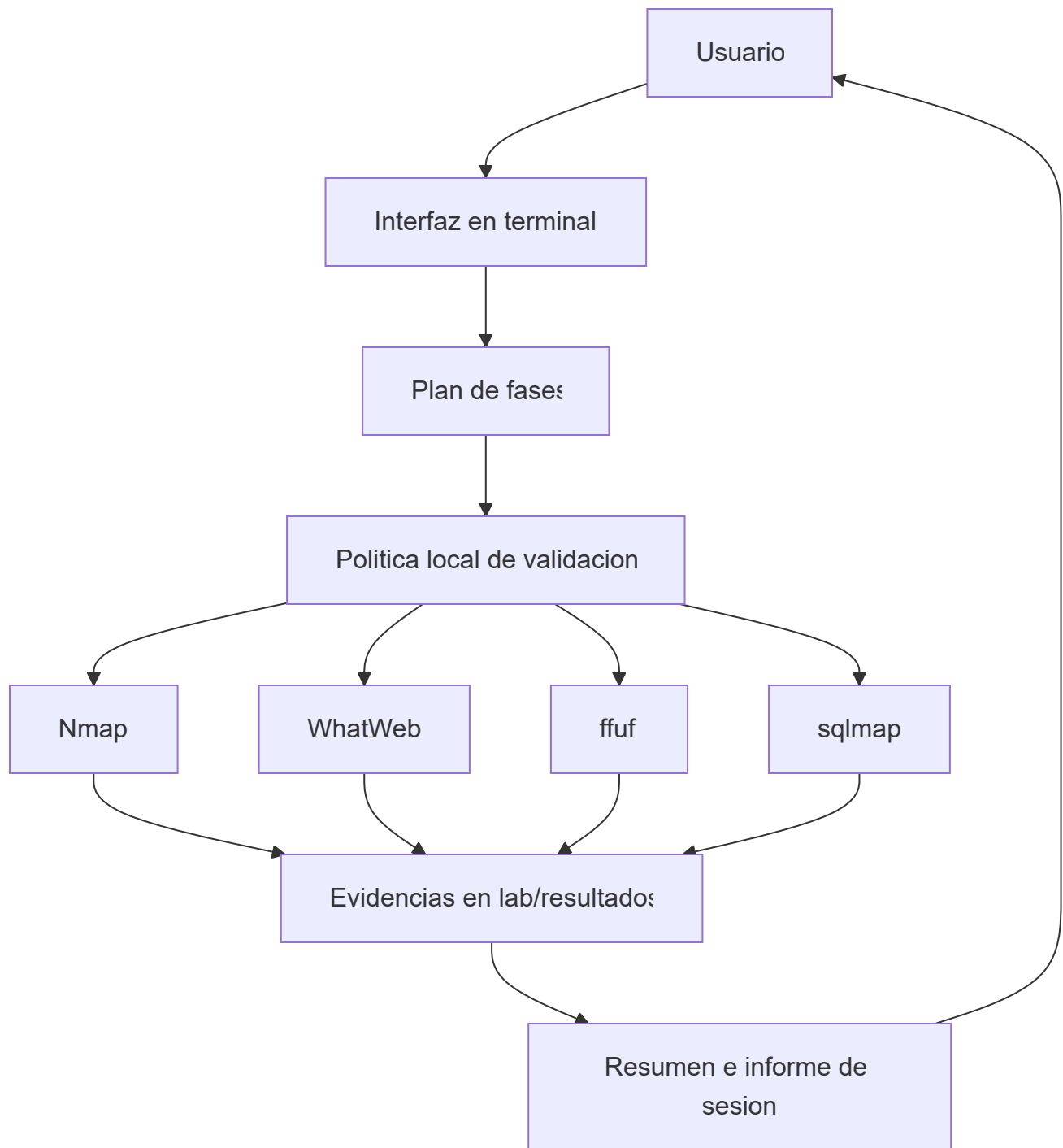
[INSERTAR CAPTURA: `lab/capturas/agente_menu_interactivo.png`, menú del agente terminal y selección del perfil `Web vulnerable`]

[INSERTAR CAPTURA: `lab/capturas/agente_plan_preparado.png`, plan preparado con objetivo, wordlist y URL vulnerable de sqlmap]

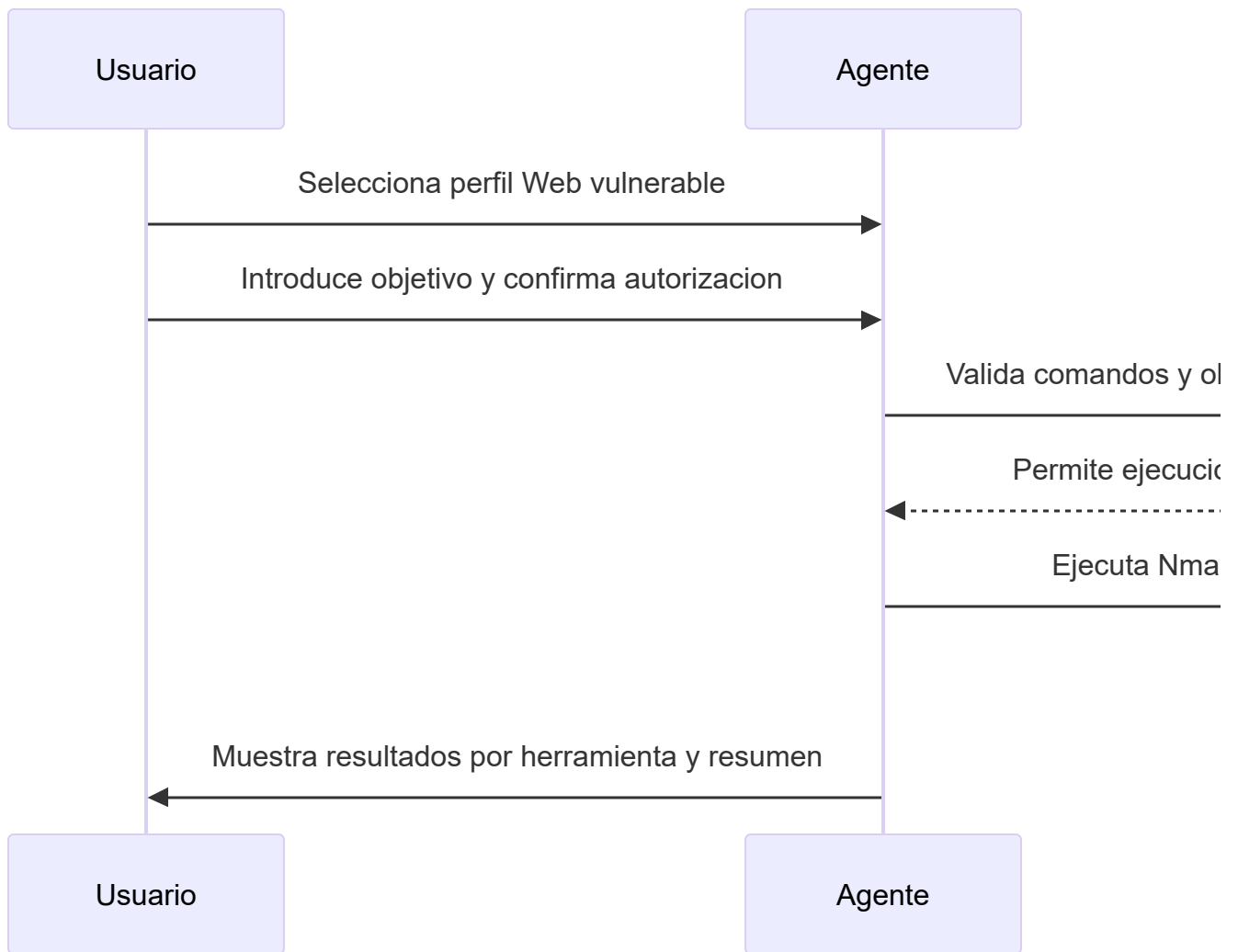
[INSERTAR CAPTURA: `lab/capturas/agente_resultados_herramientas.png`, resultados por herramienta mostrados por el agente]

[INSERTAR CAPTURA: `lab/capturas/agente_resumen_final.png`, informe global y resumen de sesión generados al finalizar]

La arquitectura ideada puede representarse de forma sencilla así:



El flujo probado en la versión alpha fue el siguiente:



El desarrollo se cerró en esta fase por limitación temporal y para mantener el centro del trabajo en el estudio de Kali Linux y sus herramientas. Durante la investigación realizada para plantear el agente se descubrió además HexStrike AI, una herramienta reciente ya recogida en el ecosistema de Kali. Ese hallazgo permitió relacionar el prototipo con una línea actual de trabajo: conectar agentes, herramientas de seguridad y ejecución controlada.

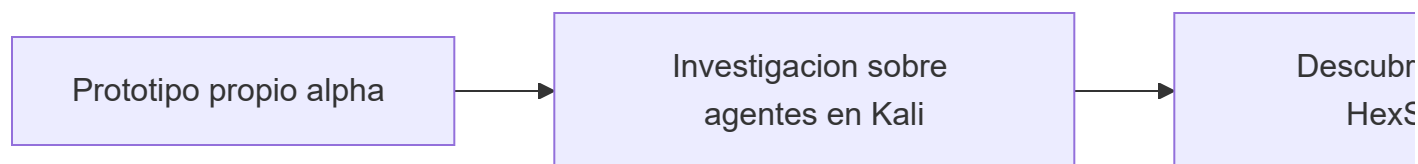
9.1 HexStrike AI como referencia encontrada durante la investigación

HexStrike AI aparece en la documentación de Kali como el paquete `hexstrike-ai`, instalable mediante `sudo apt install hexstrike-ai` (13). Kali lo describe como una plataforma de automatización de ciberseguridad basada en Model Context Protocol (MCP), con arquitectura multiagente, agentes autónomos, toma de decisiones inteligente e inteligencia de vulnerabilidades. La documentación del paquete lista dos binarios principales: `hexstrike_mcp`, orientado al cliente MCP, y `hexstrike_server`, que levanta un servidor local por defecto en `127.0.0.1:8888` (13).

El repositorio oficial de HexStrike AI lo presenta como un servidor MCP que permite conectar agentes compatibles con ese protocolo con herramientas reales de pentesting y seguridad, incluyendo reconocimiento, análisis web, explotación, forense, OSINT y generación de reportes (14). Esta descripción conecta directamente con la idea explorada en el prototipo del trabajo: usar una interfaz asistida para organizar acciones, ejecutar herramientas y transformar resultados técnicos en evidencias revisables.

En este informe, HexStrike AI se incorpora como herramienta relacionada descubierta durante la investigación. La práctica realizada se mantiene en el prototipo propio ya probado, basado en Nmap, WhatWeb, ffuf y sqlmap sobre un entorno local autorizado.

La relación entre el prototipo y este descubrimiento se resume así:



10. Limitaciones, riesgos y consideraciones éticas

La principal limitación del laboratorio es que se basa en mini escenarios locales. Esto permite trabajar con seguridad, pero no reproduce toda la complejidad de una red real ni de una aplicación completa en producción. Por ejemplo, Nmap detecta el servicio local preparado y ffuf enumera rutas creadas para la práctica.

También existen limitaciones técnicas concretas. La práctica con sqlmap se realizó contra una aplicación vulnerable mínima, por lo que demuestra la detección de una inyección SQL en un caso controlado. tcpdump requirió permisos suficientes para capturar tráfico en la interfaz de red, lo que muestra una dependencia habitual de este tipo de herramienta.

En el caso del prototipo de agente, el alcance final queda limitado a una versión alpha de terminal probada sobre un flujo web concreto. La ampliación hacia una herramienta más completa queda fuera del cierre del trabajo por tiempo. HexStrike AI se incorpora como herramienta relacionada del ecosistema Kali descubierta durante la investigación.

Desde el punto de vista ético, todas las pruebas deben realizarse sobre recursos propios o autorizados. Herramientas como Nmap, ffuf, sqlmap, tcpdump o John the Ripper pueden ser legítimas cuando existe permiso y problemáticas cuando se aplican sin control. La autorización, la minimización del alcance y la conservación responsable de evidencias son condiciones necesarias para cualquier práctica de seguridad.

11. Conclusiones

Kali Linux demuestra ser una plataforma adecuada para organizar prácticas de seguridad en ámbitos distintos. En un mismo entorno se han preparado pruebas de reconocimiento, análisis web, enumeración, explotación controlada, análisis de tráfico, auditoría de contraseñas y análisis forense básico, manteniendo una separación clara entre comandos, resultados y redacción.

Las evidencias generadas muestran que incluso un laboratorio pequeño permite explicar conceptos importantes: Nmap identifica servicios expuestos, WhatWeb reconoce tecnologías web, ffuf enumera rutas existentes, tcpdump captura tráfico en formato .pcap, John the Ripper comprueba un hash de prueba y exiftool extrae metadatos de un archivo propio. Estos resultados ilustran el papel de cada herramienta dentro de una auditoría básica.

El trabajo también muestra que algunas prácticas requieren preparación específica. sqlmap utilizó una aplicación vulnerable local creada expresamente y tcpdump requiere permisos adecuados. Los scripts facilitaron la repetición de comandos y la conservación de evidencias, mientras que el prototipo de agente actuó como incentivo personal para sostener el interés en la parte práctica del proyecto.

El prototipo de agente queda como una prueba alpha acotada, suficiente para demostrar la idea de una interfaz terminal que lanza herramientas, valida el objetivo y muestra resultados por fases. Durante esa investigación se descubrió HexStrike AI, ya presente en la documentación de herramientas de Kali, lo que permite cerrar el trabajo con una herramienta relacionada para estudiar posteriormente, manteniendo siempre autorización, trazabilidad y límites técnicos claros.

12. Bibliografía

- (1) Kali Linux. (2025). *Kali Tools*. Kali Linux Documentation. <https://www.kali.org/docs/tools/kali-tools/>
- (2) Kali Linux. (s. f.). *Kali Linux Tools*. <https://www.kali.org/tools/>
- (3) Nmap Project. (s. f.). *Nmap Documentation*. <https://nmap.org/docs.html>
- (4) Nmap Project. (s. f.). *Nmap Reference Guide*. https://nmap.org/data/nmap_manpage.html
- (5) WhatWeb Project. (2026). *WhatWeb - Next generation web scanner*. <https://github.com/urbanadventurer/WhatWeb>
- (6) ffuf Project. (s. f.). *ffuf - Fuzz Faster U Fool*. <https://github.com/ffuf/ffuf>
- (7) ffuf Project. (2023). *ffuf documentation*. <https://github.com/ffuf/ffuf/wiki>
- (8) sqlmap Project. (2025). *Usage*. <https://github.com/sqlmapproject/sqlmap/wiki/Usage>

- (9) Openwall. (s. f.). *John the Ripper documentation*. <https://www.openwall.com/john/doc/>
- (10) ExifTool. (2026). *ExifTool by Phil Harvey*. <https://exiftool.org/>
- (11) OWASP Foundation. (2026). *OWASP Juice Shop*. <https://owasp.org/www-project-juice-shop/>
- (12) Ubuntu Manpage Repository. (s. f.). *tcpdump - dump traffic on a network*. <https://manpages.ubuntu.com/manpages/focal/man8/tcpdump.8.html>
- (13) Kali Linux. (2026). *hexstrike-ai | Kali Linux Tools*. <https://www.kali.org/tools/hexstrike-ai/>
- (14) 0x4m4. (2026). *HexStrike AI MCP Agents*. <https://github.com/0x4m4/hexstrike-ai>